

CNN for MNIST Classification

Vibe Coding with GitHub Copilot / Codex

Course:	Introduction to Deep Learning
Topic:	Convolutional Neural Networks (2D CNN)
Dataset:	MNIST Handwritten Digits
Tools:	Python · PyTorch · VS Code · GitHub Copilot / Codex
Files provided:	mnist_splits.json (read-only — do not modify)

Learning Objectives

In this exercise you will use an AI coding agent (GitHub Copilot or Codex in VS Code) to build, evaluate, and iterate on a Convolutional Neural Network (CNN) for image classification. The goal is to give you a first experience of **vibe coding**: guiding an AI agent with natural-language prompts to produce working research code, while you retain the critical eye of a scientist.

By the end of this homework you will be able to:

- Set up and use GitHub Copilot / Codex as a coding agent in VS Code.
- Instruct an agent to implement a 2D CNN in PyTorch and understand the generated code.
- Load a pre-defined, reproducible train/test split and verify it is used correctly.
- Tune CNN hyperparameters and evaluate their effect on accuracy and convergence.
- Critically inspect agent-generated code for data leakage, incorrect architectures, and other pitfalls.
- Produce publication-quality plots and assemble them into a results dashboard.
- Compare deep-learning models against classical ML baselines.

Part 0 — Environment Setup

0.1 Python Environment

Create a fresh virtual environment and install the required packages. Open a terminal and run:

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install torch torchvision matplotlib scikit-learn pandas jupyter
```

0.2 Setting Up GitHub Copilot in VS Code

Follow these steps to activate the AI coding agent inside VS Code:

Step-by-step: Copilot installation

- 1. Install VS Code** — Download from <https://code.visualstudio.com> if not already installed.
- 2. Sign in to GitHub** — You need a GitHub account with Copilot access (free for students via GitHub Education).
- 3. Install the extension** — Open VS Code, go to the Extensions panel (Ctrl+Shift+X), search *GitHub Copilot*, and click Install. Also install *GitHub Copilot Chat*.
- 4. Authenticate** — A browser window will open asking you to authorise VS Code. Follow the prompts.
- 5. Enable Agent mode** — In the Copilot Chat panel, click the sparkle icon or open the Command Palette (Ctrl+Shift+P) and select *GitHub Copilot: Open Chat*. Switch the mode selector to **Agent**.
- 6. Verify** — Open any .py file, type a comment such as *# define a CNN layer* and confirm that Copilot suggests code inline.

Alternative: OpenAI Codex / VS Code Insiders

If you prefer OpenAI Codex: install the *Copilot Chat* extension and use an OpenAI API key in the advanced settings, or use the *Continue* extension with an OpenAI-compatible endpoint.

0.3 Connecting VS Code to the WEXAC Cluster

If you are working on the Weizmann Institute WEXAC HPC cluster (recommended for GPU access), you can connect your local VS Code directly to a remote WEXAC session. This lets you run and debug code on the cluster while keeping the familiar VS Code interface locally.

How to connect VS Code to WEXAC

Full instructions are available on the AIHub wiki: https://hpcwiki.weizmann.ac.il/en/home/ai_hub

Follow the section on *Opening a JupyterHub / VS Code session on WEXAC* and the instructions for setting up the **aihub_env** conda environment.

For GPU deep learning work, make sure to select a **GPU partition** when launching your session.

If you encounter any issues with the WEXAC connection or VS Code remote setup, contact **Tomer** (AIHub support) for assistance.

■ *Working on WEXAC is optional but strongly recommended if you want to train your CNN faster. On a CPU-only machine, training may take considerably longer per epoch.*

Part 1 — Loading the Pre-defined Train / Test Split

A canonical train/test split is provided in **mnist_splits.json**. Using a shared split ensures all students' results are comparable and prevents accidental data leakage. Your job here is fully agentic: you will instruct the agent to read the file and set up the data pipeline, without writing any code yourself.

■ **DO NOT MODIFY** **mnist_splits.json**. The agent must never alter or regenerate this file. If you suspect the file has been changed, re-download the original from the course repository.

Before You Start — Key Agent Tips

If you are new to working with AI coding agents, keep the following tips in mind throughout the entire homework. They are also collected for reference in **Appendix A**.

- **Be specific:** mention the file name, function name, and exact input/output shapes in your prompt.
- **Iterate:** if the first response is wrong, reply with the error message and ask it to fix it.
- **Protect files:** always include 'Do NOT modify mnist_splits.json' in any prompt that touches data.
- **Ask for comments:** request that the agent add docstrings explaining each architectural choice.
- **Verify shapes:** ask the agent to add an assert statement checking tensor shapes before/after each layer.

1.1 Setting a Persistent Agent Rule (Skill / Memory)

Before sending any coding prompt, take a moment to establish a standing rule that will protect the split file throughout the entire homework session. Most modern agent interfaces (Copilot Chat, Cursor, Continue) support some form of persistent instructions — variously called a **skill**, a **memory**, a **custom instruction**, or a **.cursorrules** / **copilot-instructions.md** file.

How to add a persistent rule in Copilot / Cursor

GitHub Copilot (VS Code): Create a file called **.github/copilot-instructions.md** at the root of your project. Any text in this file is automatically included as a standing instruction for every Copilot Chat session in that workspace.

Cursor: Go to Settings > Rules for AI and add the rule text there, or create a **.cursorrules** file at the project root.

Continue extension: Add a system-level instruction in the Continue config file (~/.continue/config.json).

See **Appendix A** for a ready-to-use rule text you can paste directly into any of these files.

Task 1

Create a **.github/copilot-instructions.md** file (or the equivalent for your agent) containing the persistent rule from **Appendix A**. Verify the agent acknowledges the rule by opening a new chat and asking: *What files are you not allowed to modify in this project?*

1.2 Agentic Data Loading

Now instruct the agent to write all data loading code for you. You do not need to write any code — describe what you need in natural language and let the agent produce the file.

Agent suggested prompt

"Read mnist_splits.json and create a file data_utils.py that: (1) loads the MNIST dataset using torchvision.datasets.MNIST, respecting the train/test flags defined in the JSON; (2) applies the normalisation mean and std found in the JSON preprocessing section; (3) keeps images as 2D tensors of shape (1, 28, 28) — do NOT flatten; (4) returns a train DataLoader (shuffled) and a test DataLoader (not shuffled), both accepting batch_size as a parameter; (5) prints the number of samples in each split and the shape of one batch to verify correctness. Do NOT modify mnist_splits.json."

Task 2

Send the prompt above to the agent and run the generated data_utils.py. Confirm the printed batch shape is **(batch_size, 1, 28, 28)** and that sample counts match the JSON (60,000 train / 10,000 test).

■ If the batch shape is wrong (e.g. flattened to 784), paste the error or incorrect shape back to the agent and ask it to fix the transform. Remember: the persistent rule from Appendix A should prevent the agent from touching the JSON, but always double-check.

Part 2 — Building a 2D CNN with the Agent

In this part you will follow a **plan-then-execute** workflow. Before asking the agent to write any code, you will first work with it in planning mode to agree on an architecture. Only once the plan is clear and validated will you instruct the agent to implement it.

2.1 Planning Mode — Design the Architecture First

Start a new Copilot Chat conversation and ask the agent to *plan* rather than to code. Here is a suggested opening prompt:

Agent suggested prompt — planning phase

"Before writing any code, describe in plain English the architecture of a 2D CNN suitable for classifying 28x28 greyscale images into 10 classes. List each layer in order, its type, output shape, and purpose. Do not write Python yet."

Once the agent replies with a plan, check it against the reference architecture in **Appendix B**. A well-formed CNN plan for MNIST should contain all of the following elements:

- One or more **Conv2d** layers (spatial feature extraction).
- **ReLU** (or similar) non-linearity after each conv layer.
- **MaxPool2d** (or similar) for spatial down-sampling.
- A **Flatten** operation transitioning from spatial to vector representation.
- One or more **Linear** (fully-connected) layers.
- An output layer with **10 units** (one per digit class).
- A **Dropout** layer for regularisation (recommended).

■ If the agent's plan does not look like a typical CNN (e.g. it suggests only fully-connected layers, or skips pooling entirely), ask it to revise before proceeding. A plan that skips Conv2d layers is not a CNN and must be corrected now. See Appendix B for a concrete reference example.

2.2 Write Your Own Prompt — Then Compare

Now, *before* sending any implementation prompt to the agent, write your own version of the coding prompt on paper or in a text file. Think about what the agent needs to know: file name, class name, input/output shapes, training loop structure, loss function, data source.

Agent suggested prompt — implementation phase

"Create a PyTorch file `cnn_model.py` containing: (1) a CNN class `MNISTConvNet` with the architecture we just planned — at least two `Conv2d` layers, `ReLU` activations, `MaxPool2d` pooling, `Dropout`, and two fully-connected layers with log-softmax output for 10 classes; (2) a `train_one_epoch(model, loader, optimizer, criterion)` function returning average loss; (3) an `evaluate(model, loader)` function returning accuracy and average loss; (4) a `main()` block that calls `load_splits()` from `data_utils.py`, trains for 10 epochs, and prints epoch-level train/test loss and accuracy. Keep images as 2D tensors — do NOT flatten before the convolutional layers. Do NOT modify `mnist_splits.json`."

Task 3	Compare your own prompt with the suggested one above. Does your version mention the file name? The input shape constraint? The data source? Refine your prompt to include any missing details, then send the refined version to the agent.
Task 4	Review the generated code: does it match the architecture plan from step 2.1? Check that Conv2d layers appear <i>before</i> any Flatten or Linear layers. Run <code>cnn_model.py</code> and confirm it trains without errors.

2.3 Understanding the Architecture

Ask the agent to help you answer the following questions. Write your answers as comments at the top of `cnn_model.py`:

- How many learnable parameters does the network have? (Ask the agent to add a parameter-count print.)
- What is the spatial size of the feature maps after each convolutional and pooling layer?
- What is the role of MaxPool2d? What would happen if you removed it?
- Why is log-softmax used instead of softmax? Which loss function must be paired with it?

Part 3 — Hyperparameter Exploration

Now instruct the agent to systematically vary architectural hyperparameters and evaluate the effect on test accuracy.

3.1 Architecture Changes

Send the following prompts to the agent **one at a time**, re-evaluating after each:

Agent suggested prompts for architecture exploration	
Prompt A — "Add a third convolutional layer with 128 filters of size 3x3 before the FC layers. Re-train and report the new test accuracy." Prompt B — "Change the number of filters in the first Conv2d from 32 to 64 and the second from 64 to 128. Does test accuracy improve?" Prompt C — "Add batch normalisation (BatchNorm2d) after each convolutional layer. Compare convergence speed with and without batch norm." Prompt D — "Try two different learning rates: 1e-2 and 1e-4. Plot the training curves for each and compare convergence."	
Task 5	Run at least two of the variants above. Record the final test accuracy for each configuration in a results table — ask the agent to build this as a pandas DataFrame and print it.

■ Each time you run a new variant, use the same `mnist_splits.json` split. If the agent suggests re-downloading or reshuffling the data, refuse and remind it of the persistent rule.

3.2 Choosing Your Best Model

Select the best-performing configuration. Ask the agent to save the trained weights:

```
torch.save(model.state_dict(), "best_cnn.pth")
```

■ This file will be loaded again in Part 5 for the final dashboard.

3.3 Bonus — Automated Hyperparameter Search with autoresearch

BONUS Ask the agent to integrate **autoresearch** (<https://github.com/karpathy/autoresearch>) to automate hyperparameter search. autoresearch is an LLM-driven research loop that proposes experiments, runs them, and iterates based on results — a form of agentic experimentation beyond manual vibe coding.

If you want to try this bonus section, first read the autoresearch README at <https://github.com/karpathy/autoresearch> to understand how it works. Then prompt your agent as follows:

Agent suggested prompt — autoresearch integration

"Install autoresearch (pip install autoresearch or clone from <https://github.com/karpathy/autoresearch>). Set up an autoresearch experiment that searches over at least three hyperparameters of MNISTConvNet: number of filters in the first conv layer, learning rate, and dropout rate. Use the train/test split from `data_utils.py`. Run the search for at least 5 iterations and report the best configuration found. Do NOT modify `mnist_splits.json`."

Task Bonus

Run the autoresearch loop. Report the best hyperparameter configuration it found and compare it to your manually tuned best model from section 3.2. Was the automated search better, worse, or similar? Reflect on the difference in effort.

Part 4 — Visualisations and Convergence Analysis

Instruct the agent to generate the following plots. Each plot should be saved as a .png file in a subfolder called **figures/**.

4.1 Loss Convergence Plot

Agent suggested prompt

"Create a function `plot_loss_curves(train_losses, test_losses, title)` that plots training and test loss vs epoch on the same axes. Save the figure to `figures/loss_curves.png`. Include a legend, axis labels, and a grid. Do NOT modify `mnist_splits.json`."

Task 6

Plot the loss curves for your baseline CNN and at least one variant from Part 3. Do the curves converge? Is there a visible gap between train and test loss suggesting overfitting?

4.2 Accuracy Comparison vs Classical ML Baselines

Agent suggested prompt

"In a new file `baselines.py`, train and evaluate two classical ML models on the same MNIST split: (1) Logistic Regression (`sklearn.linear_model.LogisticRegression`), (2) Random Forest (`sklearn.ensemble.RandomForestClassifier` with 100 trees). Use the same split from `mnist_splits.json` via `data_utils.py`. Print test accuracy for each. Do NOT modify `mnist_splits.json`."

Task 7

Compare test accuracies of: Logistic Regression, Random Forest, your baseline CNN, and your best CNN. Ask the agent to create a bar chart (`figures/accuracy_comparison.png`) showing all four.

4.3 Confusion Matrix

Agent suggested prompt

"Add a function `plot_confusion_matrix(model, test_loader)` that computes and visualises a confusion matrix for the test set using `seaborn` or `matplotlib`. Save to `figures/confusion_matrix.png`."

Task 8

Inspect the confusion matrix. Which digit pairs are most often confused? Does this make intuitive sense?

4.4 Sample Predictions

Agent suggested prompt

"Add a function `show_predictions(model, test_loader, n=10)` that displays `n` test images with their ground-truth label and predicted label. Show correct predictions in green and incorrect ones in red. Save to `figures/sample_predictions.png`."

Task 9

Run `show_predictions` and inspect the errors visually. Are they images that a human would also find ambiguous?

Part 5 — Results Dashboard

Collect all figures into a single HTML dashboard so results can be shared easily.

Agent suggested prompt

"Create a file `dashboard.py` that generates a static HTML file `dashboard.html`. The dashboard should: (1) display all `.png` files in the `figures/` folder in a 2-column grid layout; (2) include a summary table with model name, number of parameters, and test accuracy; (3) have a title section with the exercise name and date. Use only the Python standard library — no Flask, Streamlit, or other web frameworks. Do NOT modify `mnist_splits.json`."

Task 10

Run `dashboard.py` and open `dashboard.html` in a browser. Verify all four figures are visible (loss curves, accuracy comparison, confusion matrix, sample predictions).

■ If Copilot generates a Flask or Streamlit server, ask it to produce a static HTML file instead — it is simpler to share.

Part 6 — Critical Review of Agent-Generated Code

A key skill for AI-assisted development is knowing when to *trust* and when to *question* the agent. This part asks you to audit the code that was generated.

6.1 Data Usage Audit

Your first option is to read through the generated code directly. Inspect `data_utils.py`, `cnn_model.py`, and `baselines.py` and look for the following potential issues:

- **Split integrity:** Were training and test sets ever mixed? Look for any place where test data could have leaked into training.
- **Input format:** Confirm the CNN receives images of shape (N, 1, 28, 28). If the agent added Flatten before the convolutional layers, that is a bug.
- **Normalisation:** Were the normalisation statistics computed only on the training set, or incorrectly on both splits?
- **Shuffle:** Is the training loader shuffled? Is the test loader unshuffled?

If the code is too complex to read through comfortably — because it is long, because you are less experienced with PyTorch internals, or simply because you do not have time — you have an equally valid alternative: **ask the agent to summarise the workflow for you**.

Agent suggested prompt — code summary

"Read all the Python files in this project (`data_utils.py`, `cnn_model.py`, `baselines.py`) and write a plain-English summary of the data pipeline in a file called `CODE_SUMMARY.md`. For each file, describe: (1) where the data comes from and which split is used for which purpose; (2) what preprocessing is applied and at what stage; (3) whether training and test data are ever combined or used interchangeably; (4) the shape of the data at each major step. Flag any potential data leakage or preprocessing inconsistencies explicitly."

Task 11

Either review the code directly OR ask the agent to generate `CODE_SUMMARY.md` and review that instead. Write a short paragraph (3–5 sentences) in `AUDIT.md` confirming whether the data pipeline is correct, and describing any issues you found and how they were fixed.

6.2 ML Model Audit — Identifying What Was Used

Inspect `baselines.py` and `cnn_model.py` and answer the following questions in `AUDIT.md`:

- Which scikit-learn estimator(s) did the agent choose for the classical baselines? Are they appropriate for image classification?
- For the CNN: which optimiser was used? Which loss function? Were these the correct choices?
- Did the agent use any pre-trained models or transfer learning? If so, was that appropriate for this exercise?

6.3 Redirecting the Agent to a Specific Model — TabICLv2

Now you will ask the agent to replace the Logistic Regression baseline with a more recent tabular model: **TabICL v2**. TabICL is an in-context learning model for tabular data that treats classification as few-shot learning without gradient-based training.

Agent suggested prompt

"In `baselines.py`, replace the Logistic Regression model with TabICLv2 from the `tabicl` library (GitHub: <https://github.com/soda-inria/tabicl> — install with: `pip install tabicl`). Flatten the MNIST images to 784-dimensional feature vectors for the tabular model input. Use the same train/test split loaded via `data_utils.py` from `mnist_splits.json`. Report test accuracy and add TabICLv2 as a new bar in the accuracy comparison chart (`figures/accuracy_comparison.png`). Do NOT modify `mnist_splits.json`."

Task 12

Run the updated `baselines.py` with TabICLv2. Does it outperform Logistic Regression? Is it competitive with your CNN? Add the result to `AUDIT.md`.

■ *TabICL v2 reference: Zhu et al. (2025). Available at <https://github.com/soda-inria/tabicl>*

■ TabICL operates on flattened feature vectors — it is a tabular model and does NOT exploit the 2D spatial structure of images. This is an intentional contrast with the CNN.

Part 7 — Reflection Questions

Answer the following questions in `AUDIT.md`. Each answer should be 3–6 sentences.

- **Q1.** How did vibe coding change your workflow compared to writing code from scratch? What were the main benefits and risks?
- **Q2.** The CNN significantly outperforms Logistic Regression. What structural property of CNNs explains this advantage on image data?
- **Q3.** What happened to the loss curves when you increased the number of filters (Part 3)? Did more capacity always help?
- **Q4.** TabICL v2 treats pixel values as a flat feature vector. What information does it lose compared to the CNN, and how does this affect accuracy?
- **Q5.** If you were deploying this model in a real application, what additional checks (beyond test accuracy) would you perform?

Submission Checklist

Submit a zip archive named **lastname_firstname_cnn_hw.zip** containing:

■	<code>mnist_splits.json</code>	Unchanged from the provided version
■	<code>.github/copilot-instructions/agent_rule.json</code>	Persistent agent rule file (or equivalent)
■	<code>data_utils.py</code>	Agentic data loading utilities
■	<code>cnn_model.py</code>	CNN architecture and training loop
■	<code>baselines.py</code>	Classical ML baselines (including TabICLv2)
■	<code>dashboard.py</code>	Dashboard generator
■	<code>dashboard.html</code>	Rendered results dashboard
■	<code>AUDIT.md</code>	Data audit + reflection answers (Parts 6-7)
■	<code>CODE_SUMMARY.md</code>	Agent-generated code summary (if used in 6.1)
■	<code>figures/</code>	All 4 saved figures as .png
■	<code>best_cnn.pth</code>	Saved weights of your best model

Appendix — Quick Reference

A. Key Copilot / Codex Agent Tips

- **Be specific:** mention the file name, function name, and exact input/output shapes in your prompt.
- **Iterate:** if the first response is wrong, reply with the error message and ask it to fix it.
- **Protect files:** always include 'Do NOT modify mnist_splits.json' in any prompt that touches data.
- **Ask for comments:** request that the agent add docstrings explaining each architectural choice.
- **Verify shapes:** ask the agent to add an assert statement checking tensor shapes before/after each layer.

B. CNN Architecture — Typical Building Blocks

The reference architecture below illustrates the expected layer order for a standard 2D CNN on MNIST. Use this to validate the agent's plan in Part 2.1.

```
class MNISTConvNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1), # (N,1,28,28) ->
(N,32,28,28)
            nn.ReLU(),
            nn.MaxPool2d(2), # (N,32,28,28) ->
(N,32,14,14)
            nn.Conv2d(32, 64, kernel_size=3, padding=1), # (N,32,14,14) ->
(N,64,14,14)
            nn.ReLU(),
            nn.MaxPool2d(2), # (N,64,14,14) -> (N,64,7,7)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(), # (N,64,7,7) -> (N,3136)
            nn.Linear(64 * 7 * 7, 128),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(128, 10),
            nn.LogSoftmax(dim=1),
        )
    def forward(self, x):
        return self.classifier(self.features(x))
# Pair with: criterion = nn.NLLLoss()
```

■ *Flatten must appear AFTER all Conv2d and pooling layers — never before. Placing Flatten before Conv2d would turn the CNN into an MLP and discard all spatial information.*

C. VS Code + Copilot Agent — Cheat Sheet

The table below covers the most useful keyboard shortcuts and agent interaction patterns for GitHub Copilot in VS Code. For complete documentation, see <https://docs.github.com/en/copilot>.

Shortcut / Action	What it does
Ctrl + Shift + P	Open Command Palette — type 'Copilot' to find all agent commands
Ctrl + Shift + I	Open / focus the Copilot Chat panel
Ctrl + I (in editor)	Inline edit — select code, press Ctrl+I, describe the change in plain English
Ctrl + Enter	Accept an inline Copilot autocomplete suggestion
Alt +] / Alt + [	Cycle forward / backward through alternative Copilot suggestions
Esc	Dismiss the current Copilot suggestion without accepting
@workspace (in Chat)	Ask Copilot to reason over all open files in the project
#file:name (in Chat)	Attach a specific file as context for the current Chat message
/explain (in Chat)	Ask Copilot to explain selected code in plain English
/fix (in Chat)	Ask Copilot to diagnose and fix an error — paste the error message after /fix
/tests (in Chat)	Ask Copilot to generate unit tests for selected code
Agent mode toggle	In Chat panel: switch the dropdown from 'Ask' to 'Agent' for multi-step agentic tasks

Additionally, here are recommended prompt patterns that work well for deep learning workflows:

Pattern	Example prompt fragment
Shape assertion	"Add an assert to check the tensor shape is (N,1,28,28) after loading"
Parameter count	"Print the total number of trainable parameters in the model"
Explain a layer	"Explain in a comment what MaxPool2d(2) does to the feature map dimensions"
Fix with error	"I got this error: [paste error]. Fix it without changing the split logic"
Summarise code	"Summarise this file in 5 bullet points, focusing on data flow"
Protect file (reminder)	"Reminder: do NOT modify mnist_splits.json"
Architecture validation	"Does this architecture qualify as a CNN? What is missing?"
Refactor for clarity	"Refactor this function to be more readable, keep behaviour identical"

D. References

- LeCun et al. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*.
- PyTorch documentation: <https://pytorch.org/docs/stable/index.html>
- GitHub Copilot documentation: <https://docs.github.com/en/copilot>
- Zhu et al. (2025). TabICL: Scaling Tabular Learning with In-Context Learning. <https://github.com/soda-inria/tabicl>

- Karpathy, A. autoresearch — LLM-driven automated research loop.
<https://github.com/karpathy/autoresearch>
- MNIST dataset: <http://yann.lecun.com/exdb/mnist/>
- AIHub wiki (WEXAC cluster, VS Code remote setup):
https://hpcwiki.weizmann.ac.il/en/home/ai_hub